Proyecto 2 — Algoritmo CYK

Simplificación de gramáticas libres de contexto y parsing

Harry Mendez (24089)
Juan Gualim (24852)
Diego Parada (2020225)

Teoría de la Computación
Universidad del Valle de Guatemala

Octubre 2026

Repositorio: `https://github.com/hmndz3/Proyecto2TC`

Índice

1. Introducción **3**

2. Marco: gramáticas y lenguajes **3**
2.1. Qué es una gramática . . . 3
2.2. La gramática del enunciado . . . 3
2.3. Por qué hace falta la Forma Normal de Chomsky . . . 4

3. Diseño de la aplicación **4**
3.1. Por qué Elixir . . . 4
3.2. Organización en módulos . . . 4
3.3. Flujo del programa . . . 5
3.4. Formato de entrada . . . 5

4. El algoritmo de simplificación **6**
4.1. Paso 1: eliminación de producciones épsilon . . . 6
4.2. Paso 2: eliminación de producciones unitarias . . . 6
4.3. Paso 3: eliminación de símbolos inútiles . . . 7
4.4. Paso 4: Forma Normal de Chomsky . . . 7

5. El algoritmo CYK **8**
5.1. La idea . . . 8
5.2. Qué es la programación dinámica . . . 9
5.3. La tabla, con un ejemplo . . . 9
5.4. Backpointers y el árbol . . . 9
5.5. Complejidad . . . 10

6. Cómo ejecutar el programa **10**

7. Ejemplos y pruebas **11**
7.1. Frases que sí están en el lenguaje . . . 11
7.2. Frases que no están en el lenguaje . . . 12
7.3. Expresiones aritméticas . . . 12
7.4. Ambigüedad . . . 12

7.5. Tiempos de ejecución 12
7.6. Pruebas automáticas 12

8. Discusión **13**
8.1. Obstáculos encontrados 13
8.2. Recomendaciones 14

9. Conclusiones **14**

10. Referencias **14**

1. Introducción

Este proyecto responde a una pregunta sencilla de enunciar y difícil de resolver: **dada una gramática y una frase, ¿esa frase pertenece al lenguaje que la gramática genera?**

Para una gramática pequeña uno puede intentar derivar la frase a mano. Pero ese método tiene un problema: si la gramática es ambigua o tiene recursión, uno puede quedarse probando derivaciones para siempre sin llegar a una respuesta. El algoritmo **CYK** (Cocke–Younger–Kasami) resuelve esto: siempre termina, siempre da una respuesta correcta, y lo hace en tiempo polinomial.

El precio que cobra CYK es que la gramática tiene que estar en **Forma Normal de Chomsky** (CNF): toda producción debe ser de la forma $A \to BC$ o $A \to a$. Casi ninguna gramática viene así escrita, por eso la primera mitad del proyecto es convertirla, y esa conversión tiene sus propias reglas que hay que aplicar en orden.

Entonces el programa hace dos cosas:

1. **Simplifica** la gramática y la lleva a CNF, en cuatro pasos.
2. **Valida** una frase con CYK, reportando sí o no, cuánto tardó y el árbol de derivación.

Está escrito en **Elixir**, sin librerías externas, y se usa desde la terminal.

2. Marco: gramáticas y lenguajes

2.1. Qué es una gramática

Una gramática libre de contexto se describe con cuatro cosas, $G = (V, T, P, S)$:

- V: las **variables** o símbolos no terminales. Son los que se pueden seguir reemplazando. Por convención van en mayúscula: `S`, `NP`, `VP`.
- T: los **terminales**. Son las palabras reales del lenguaje, las que ya no se reemplazan por nada: `she`, `eats`, `id`, `+`.
- P: las **producciones** o reglas, del tipo `A -> B C`.
- S: el **símbolo inicial**, por donde empieza toda derivación.

«Libre de contexto» significa que el lado izquierdo de cada regla es siempre una sola variable. No existe algo como `a B -> c`: la variable `B` se reemplaza igual sin importar qué tenga a los lados. Eso es lo que la distingue de las gramáticas sensibles al contexto en la jerarquía de Chomsky, y lo que hace que exista un algoritmo eficiente para reconocerlas.

2.2. La gramática del enunciado

```
S   -> NP VP
VP  -> VP PP | V NP | cooks | drinks | eats | cuts
PP  -> P NP
NP  -> Det N | he | she
V   -> cooks | drinks | eats | cuts
P   -> in | with
N   -> cat | dog | beer | cake | juice | meat | soup | fork | knife | oven | spoon
Det -> a | the
```

Leída en voz alta: una oración (`S`) es un sujeto (`NP`) seguido de un predicado (`VP`). Un `NP` es un determinante más un sustantivo (`the cat`) o un pronombre (`he`, `she`). Un `VP` es un verbo con su objeto (`eats a cake`), o un `VP` con una frase preposicional pegada (`eats a cake with a fork`), o un verbo solo (`cooks`).

2.3. Por qué hace falta la Forma Normal de Chomsky

CYK funciona partiendo la frase en dos pedazos y preguntando, para cada partición, qué variable puede generar el pedazo izquierdo y cuál el derecho. Eso solo tiene sentido si las reglas tienen exactamente dos símbolos del lado derecho. Una regla como `S -> A B C d` no dice dónde partir.

Por eso toda producción debe quedar de una de estas dos formas:

$A \to B\,C$	dos variables, ninguna terminal
$A \to a$	una sola terminal

Y además no puede haber producciones vacías ($A \to \varepsilon$) ni unitarias ($A \to B$), porque romperían el conteo de longitudes en que se basa el algoritmo.

3. Diseño de la aplicación

3.1. Por qué Elixir

El enunciado no pide ningún lenguaje en particular. Elegimos Elixir por tres razones concretas:

1. **Los tres pasos de simplificación son cálculos de punto fijo.** Todos tienen la misma forma: se aplica una regla sobre un conjunto una y otra vez, hasta que el conjunto deja de crecer. En un lenguaje funcional eso se escribe como una función recursiva corta que compara el conjunto anterior con el nuevo y se detiene cuando son iguales. No hacen falta variables mutables ni banderas de control.

2. **Los conjuntos y los mapas vienen con el lenguaje.** Elixir trae `MapSet` y `Map` listos para usar, y resultan ser justo las dos estructuras que este problema necesita: conjuntos para los anulables, los generadores y los alcanzables; mapas para las reglas de la gramática y para la tabla de CYK. Operaciones como «agregar al conjunto si todavía no está» o «buscar todas las reglas que tengan este par» quedan en una línea, sin tener que programar la estructura de datos.

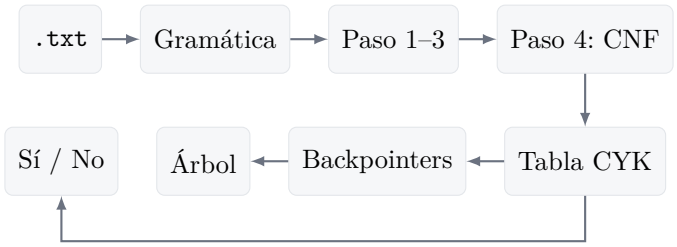
3. **No necesita instalación de dependencias.** Todo el proyecto usa solo la librería estándar.

3.2. Organización en módulos

La aplicación está separada en dos capas. Los módulos de abajo son funciones puras: reciben una gramática y devuelven otra, sin tocar la pantalla ni archivos. Los de arriba solo se encargan de mostrar.

Archivo	Responsabilidad
Lógica (funciones puras)	
`lib/cyk/grammar.ex`	Lee el formato `.txt`, guarda la gramática, la imprime de vuelta.
`lib/cyk/simplify.ex`	Pasos 1 a 3: épsilon, unitarias, inútiles.
`lib/cyk/cnf.ex`	Paso 4: Forma Normal de Chomsky.
`lib/cyk/parser.ex`	Algoritmo CYK, tabla dinámica, medición de tiempo.
`lib/cyk/tree.ex`	Reconstruye el árbol desde los backpointers.
`lib/cyk.ex`	Encadena los cuatro pasos y parte la frase en tokens.
Presentación	
`lib/cyk/ui.ex`	Formato de la salida en terminal: cajas, colores, tablas.
`lib/cyk/svg.ex`	Dibuja el árbol y lo abre en una ventana aparte.
`lib/cyk/cli.ex`	Interfaz de línea de comandos y modo interactivo.
`lib/mix/tasks/cyk.ex`	Registra el comando `mix cyk`.

3.3. Flujo del programa



3.4. Formato de entrada

Mantuvimos el mismo formato de los archivos de referencia (`1.txt` y `1-cnf.txt`), para que el programa pueda leerlos tal cual:

```
# los comentarios empiezan con numeral
S   -> NP VP
VP  -> VP PP | V NP | eats
X   -> + T X | e
```

Las reglas del formato son:

- `->` separa la cabeza del cuerpo; `|` separa alternativas.
- **Los símbolos van separados por espacios.** Por eso `id` es un solo token y `( E )` son tres. Lo mismo al escribir la frase: `( id + id ) * id`.
- `e` representa épsilon.
- Un símbolo es **no terminal** si aparece como cabeza de alguna producción. Todo lo demás es terminal. Esto es más robusto que fijarse en mayúsculas y permite nombres como `A'`.
- El **símbolo inicial** es la cabeza de la primera producción, que es la convención habitual.

Las gramáticas incluidas están en la carpeta `gramaticas/`:

Archivo	Contenido
`ingles.txt`	La del enunciado del proyecto.
`aritmetica.txt`	Expresiones aritméticas: el archivo `1.txt` de referencia.
`aritmetica-cnf.txt`	El CNF de referencia (`1-cnf.txt`), usado en las pruebas.
`ambigua.txt`	`E -> E + E | E * E | ( E ) | id`, para ver el conteo de árboles.

4. El algoritmo de simplificación

Los cuatro pasos se aplican en este orden, que es el que se vio en clase. El orden importa: por ejemplo, eliminar épsilon *crea* producciones unitarias nuevas, así que el paso 2 tiene que ir después del paso 1.

Para ilustrarlos usamos la gramática aritmética, porque la del enunciado —como se explica más adelante— ya viene en CNF y no ejercita ningún paso.

```
E -> T X
X -> + T X | e
T -> F Y
Y -> * F Y | e
F -> ( E ) | id
```

4.1. Paso 1: eliminación de producciones épsilon

Qué busca. Las producciones que generan la cadena vacía. Estorban porque CYK razona sobre longitudes de subcadena, y una variable que puede desaparecer rompe esa cuenta.

Cómo funciona. Primero se calcula el conjunto de **anulables**: toda variable que tenga una producción vacía, más toda variable cuyo cuerpo esté formado únicamente por anulables. Se repite hasta que el conjunto no crezca. Después, por cada cuerpo, se generan todas las versiones posibles quitando o dejando cada símbolo anulable, y se descartan los cuerpos que quedaron vacíos.

En este ejemplo. Anulables: `X`, `Y`.

```
E -> T X | T
X -> + T X | + T
T -> F Y | F
Y -> * F Y | * F
F -> ( E ) | id          8 -> 10 producciones
```

Un detalle: si el símbolo inicial resultaba anulable, significa que la cadena vacía sí pertenece al lenguaje. Como esa información se perdería al borrar las épsilon, el programa la guarda aparte en una bandera de la gramática.

Dónde está. `lib/cyk/simplify.ex`, funciones `epsilon/1` y `nullable/1`.

4.2. Paso 2: eliminación de producciones unitarias

Qué busca. Las reglas del tipo `A -> B`, donde una variable simplemente se renombra como otra. No aportan estructura y CNF no las permite.

Cómo funciona. Se arma el conjunto de **pares unitarios** (A, B) tales que $A \Rightarrow^* B$ usando solo reglas unitarias. Se arranca con los reflexivos (A, A) y se cierra transitivamente. Luego, a cada A se le copian las producciones **no** unitarias de cada B de su par.

En este ejemplo. Pares: `(E,F)`, `(E,T)`, `(T,F)` además de los reflexivos.

```
E -> T X | ( E ) | id | F Y
X -> + T X | + T
T -> ( E ) | id | F Y
Y -> * F Y | * F
F -> ( E ) | id          10 -> 13 producciones
```

Dónde está. `lib/cyk/simplify.ex`, funciones `unit/1` y `unit_pairs/1`.

4.3. Paso 3: eliminación de símbolos inútiles

Qué busca. Variables que no sirven para nada. Son de dos tipos y se eliminan en ese orden:

1. **No generadores**: variables que nunca llegan a producir una cadena de puros terminales. Por ejemplo `C -> a C b` sin otra alternativa: siempre queda una `C` pendiente.

2. **No alcanzables**: variables a las que nunca se llega desde el símbolo inicial.

El orden importa. Si se quitan primero los no alcanzables y después los no generadores, puede quedar basura: eliminar una variable no generadora puede dejar a otras sin forma de alcanzarse.

En este ejemplo. No hay ninguno de los dos tipos, la gramática queda igual (13 producciones). Donde sí se nota es en un caso como el siguiente:

```
Original:                    Despues de los pasos 1 y 2:
  S -> a S | A | C             S -> a S | a | a C b
  A -> a                       A -> a
  B -> a a                     B -> a a
  C -> a C b                   C -> a C b
```

```
C no genera terminales      -> se elimina C (y la regla S -> a C b)
A y B ya no son alcanzables -> se eliminan

resultado:  S -> a S | a
```

Dónde está. `lib/cyk/simplify.ex`, funciones `useless/1`, `generating/1` y `reachable/1`.

4.4. Paso 4: Forma Normal de Chomsky

Qué busca. Dejar toda producción como $A \to BC$ o $A \to a$. Son dos sub-pasos:

1. **Sacar los terminales de los cuerpos largos.** Un cuerpo como `( E )` mezcla terminales con variables. Cada terminal se reemplaza por una variable nueva que solo produce ese terminal: `T1 -> (`.

2. **Binarizar.** Un cuerpo de tres o más símbolos se parte en cadena: `A -> B C D` se vuelve `A -> B X1` junto con `X1 -> C D`.

En este ejemplo. Variables nuevas: `T1`→`(`, `T2`→`)`, `T3`→`+`, `T4`→`*`, `X1`→`E T2`, `X2`→`T X`, `X3`→`F Y`.

```
E  -> T X | T1 X1 | id | F Y
X  -> T3 X2 | T3 T
T  -> T1 X1 | id | F Y
Y  -> T4 X3 | T4 F
F  -> T1 X1 | id
T1 -> (        T2 -> )        T3 -> +        T4 -> *
X1 -> E T2     X2 -> T X      X3 -> F Y

                                              13 -> 20 producciones
```

Comparación con el CNF de referencia. Contamos con el archivo `1-cnf.txt`, que trae la conversión esperada. Nuestro resultado tiene las mismas 20 producciones y la misma estructura; solo cambian los nombres de las variables nuevas, que son arbitrarios:

Referencia	L	R	P	M	Z	W	V
Nuestro	T1	T2	T3	T4	X1	X2	X3

Dónde está. `lib/cyk/cnf.ex`.

```
── Simplificación ──────────────────────────────────────────────────────


  Paso 1 · Eliminación de producciones ε
    Anulables          X, Y
    Producciones       8 → 10

  Paso 2 · Eliminación de producciones unitarias
    Pares unitarios    (E,E) (E,F) (E,T) (F,F) (T,F) (T,T) (X,X) (Y,Y)
    Producciones       10 → 13

  Paso 3 · Eliminación de símbolos inútiles
    No generadores     (ninguno)
    No alcanzables     (ninguno)
    Producciones       13 (sin cambios)

  Paso 4 · Forma Normal de Chomsky
    Variables nuevas   T1→(, T2→), T3→+, T4→*, X1→E T2, X2→T X, X3→F Y
    Producciones       13 → 20

    Tiempo total       5.4270 ms
    ¿Es CNF válida?    sí
```

Figura 1: `mix cyk gramaticas/aritmetica.txt --pasos` — los cuatro pasos sobre la gramática aritmética, con el conteo de producciones de cada uno y la gramática resultante en CNF.

5. El algoritmo CYK

5.1. La idea

CYK es programación dinámica. En vez de intentar derivar la frase de arriba hacia abajo, construye la respuesta de abajo hacia arriba: primero resuelve las subcadenas de una palabra, después las de dos, y así hasta cubrir la frase completa.

La pregunta que responde cada celda de la tabla es: *¿qué variables pueden generar esta subcadena?*

- **Subcadenas de longitud 1.** Para la palabra `cake`, se buscan todas las variables con una regla `A -> cake`. Aquí: `N`.
- **Subcadenas más largas.** Se prueban todas las formas de partir la subcadena en dos. Para cada partición, si la mitad izquierda la puede generar `B`, la derecha la puede generar `C`, y existe una regla `A -> B C`, entonces `A` puede generar la subcadena entera.

Al final, la frase pertenece al lenguaje si y solo si el **símbolo inicial** aparece en la celda que cubre la frase completa.

5.2. Qué es la programación dinámica

La programación dinámica es una técnica para problemas que cumplen dos condiciones: se pueden partir en subproblemas más chicos, y esos subproblemas **se repiten**. La idea es resolver cada uno una sola vez y guardar el resultado en una tabla, en vez de recalcularlo cada vez que hace falta.

CYK cumple las dos. El subproblema es «¿qué variables generan la subcadena que empieza en i y mide ℓ palabras?», y se repite constantemente: al validar *she eats a cake with a fork*, el pedazo *a cake* se necesita para armar *eats a cake*, y otra vez para *eats a cake with a fork*. Resolverlo cada vez por separado llevaría a un costo exponencial; guardándolo, el algoritmo queda en $\mathcal{O}(n^3 \cdot |G|)$.

Hay dos formas de implementarla. La **memoización** es recursión que guarda lo que ya calculó, de arriba hacia abajo. La **tabulación** llena la tabla directamente de abajo hacia arriba, en un orden que garantiza que cada celda encuentre listas las que necesita. Nosotros usamos la segunda: la tabla se recorre por longitud creciente, y cada celda consulta únicamente celdas de filas anteriores. En el código es un `Map` indexado por `{posición, longitud}` que se va pasando de una iteración a la siguiente.

5.3. La tabla, con un ejemplo

Para *she eats a cake with a fork*, con las filas numeradas por longitud de subcadena:

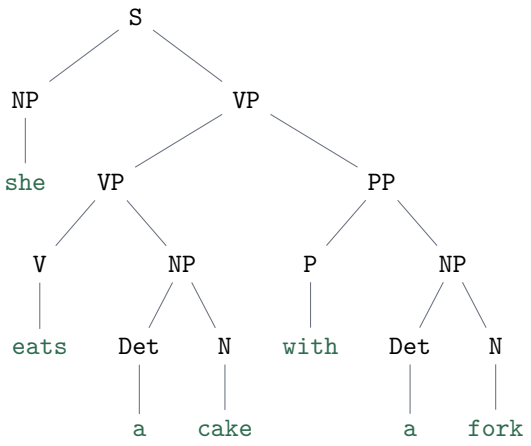
	she	eats	a	cake	with	a	fork
1	NP	V,VP	Det	N	P	Det	N
2	S	·	NP	·	·	NP	
3	·	VP	·	·	PP		
4	S	·	·	·			
5	·	·	·				
6	·	VP					
7	S						

Se lee así: la celda de la fila 2, columna `a`, contiene `NP`, lo que dice que *a cake* puede generarse desde `NP`. La celda de abajo a la izquierda (fila 7, columna `she`) cubre la frase entera y contiene `S`, el símbolo inicial. Por lo tanto la frase **sí** pertenece al lenguaje.

5.4. Backpointers y el árbol

Si cada celda guardara solo «sí, esta variable puede», la respuesta sería un booleano y nada más. Para poder construir el árbol, cada entrada guarda además **de dónde salió**: con qué regla y en qué punto se partió la subcadena. Eso es un *backpointer*.

Reconstruir el árbol es entonces seguir esos punteros hacia atrás desde la celda final. El resultado para la frase del ejemplo:



El programa muestra este árbol de dos maneras: dibujado en texto dentro de la terminal, y dibujado como gráfico en una ventana aparte que se abre automáticamente.

Los backpointers sirven para una cosa más: **contar cuántos árboles distintos existen**, que es exactamente la definición de ambigüedad. Si una celda tiene dos backpointers para la misma variable, hay dos derivaciones.

5.5. Complejidad

Son tres bucles anidados —longitud, posición inicial, punto de partición— y dentro de cada uno se recorren las reglas. Eso da $\mathcal{O}(n^3 \cdot |G|)$, donde n es la cantidad de palabras y $|G|$ el tamaño de la gramática.

Para las frases de este proyecto eso es nada: una frase de siete palabras se resuelve en unos 14 microsegundos.

Dónde está. `lib/cyk/parser.ex` (tabla y conteo de árboles), `lib/cyk/tree.ex` (reconstrucción), `lib/cyk/svg.ex` (dibujo).

6. Cómo ejecutar el programa

Lo único que hay que tener instalado es Elixir 1.14 o superior. El proyecto no usa librerías externas, así que no hay dependencias que descargar.

```
mix cyk                              # modo interactivo
mix cyk gramaticas/aritmetica.txt    # otra gramatica
mix cyk --frase "she eats a cake"    # valida una frase y sale
mix cyk --pasos                      # muestra cada gramatica intermedia
mix cyk --sin-ventana                # no abre el arbol en el navegador
mix cyk --repetir 1000               # fija las corridas de la medicion
mix test                             # las pruebas automaticas
```

Dentro del modo interactivo se pueden usar los comandos `:gramatica`, `:cnf`, `:pasos` y `:salir`.

La salida muestra, en orden: el resumen de la gramática, los cuatro pasos de simplificación con cuántas producciones entran y salen de cada uno, la gramática en CNF, el veredicto, el tiempo, la cantidad de árboles, la tabla CYK y el árbol de derivación.

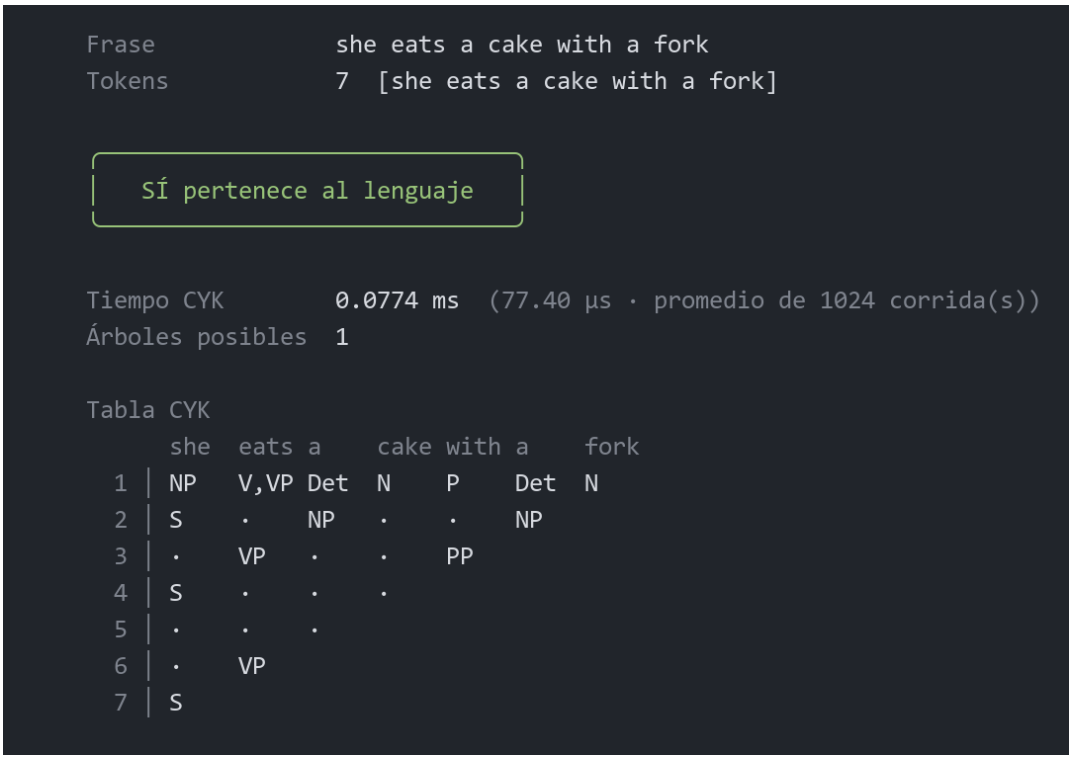


Figura 2: `mix cyk --frase "she eats a cake with a fork"` — el veredicto, el tiempo de validación, la cantidad de árboles y la tabla CYK. Debajo de esto el programa imprime además el árbol de derivación de la sección 5.3.

7. Ejemplos y pruebas

7.1. Frases que sí están en el lenguaje

Frase	Por qué
`she eats a cake with a fork`	`NP VP`, con `VP -> VP PP`.
`the cat drinks the beer`	Los dos `NP` son `Det N`.
`he drinks the beer`	`NP` pronombre.
`she cooks`	`VP` de un solo verbo.
`a dog cuts the meat with a knife`	El caso más largo, siete palabras.
`he eats the soup in the oven`	Usa la otra preposición.
`the cake eats`	Correcta sintácticamente aunque absurda.

El último caso merece un comentario. *The cake eats* se acepta porque `the cake` es un `NP` válido y `eats` es un `VP` válido. La gramática no sabe nada de qué cosas pueden comer. El enunciado lo advierte explícitamente: las frases aceptadas son sintácticamente correctas, nadie garantiza que lo sean semánticamente.

7.2. Frases que no están en el lenguaje

Frase	Por qué falla
`eats she a cake`	Empieza con verbo; `S` exige `NP` primero.
`a cat with the dog`	Es `NP PP`, le falta el `VP`.
`she eats a`	`a` es `Det` y queda sin sustantivo.
`cat the drinks beer`	`cat` suelto es `N`, no `NP`.
`she eats a cake with`	La preposición queda colgando sin `NP`.
`banana`	No es terminal de la gramática.

En el último caso el programa además avisa `Desconocidos: banana`, para distinguir «la frase está mal armada» de «esa palabra no existe en la gramática».

7.3. Expresiones aritméticas

Con `gramaticas/aritmetica.txt`. Recordar que los símbolos van separados por espacios.

Acepta		Rechaza	
`id`	sí	`+ id`	no
`id + id`	sí	`id +`	no
`( id )`	sí	`( id`	no
`( id + id ) * id`	sí	`id id`	no
`id * ( id + id )`	sí	`* id *`	no

7.4. Ambigüedad

Con `gramaticas/ambigua.txt`, que contiene `E -> E + E | E * E | ( E ) | id`. Esta gramática no define precedencia, así que una misma expresión admite varias lecturas:

Expresión	Árboles
`id + id`	1
`id + id * id`	2
`id + id * id + id`	5

Los números 1, 2, 5 son los números de Catalan, que es justo la cantidad de formas de parentizar una expresión. El programa reporta esta cuenta junto al veredicto.

7.5. Tiempos de ejecución

Medidos en la máquina de desarrollo, con la gramática del enunciado:

Frase	Palabras	Tiempo
`she cooks`	2	7.9 µs
`cat the drinks beer`	4	13.4 µs
`she eats a cake with a fork`	7	13.8 µs

7.6. Pruebas automáticas

El proyecto incluye **41 pruebas** que corren con `mix test`. Están en la carpeta `test/`:

Archivo	Qué verifica
`grammar_test.exs`	Lectura del formato, comentarios, épsilon, símbolo inicial.
`simplify_test.exs`	Los tres pasos, contra gramáticas pequeñas resueltas a mano paso por paso.
`cnf_test.exs`	Que toda producción quede $A \to BC$ o $A \to a$, y la equivalencia con el CNF de referencia.
`cyk_test.exs`	Frases aceptadas y rechazadas, conteo de árboles, forma del árbol.

La prueba de equivalencia merece explicación. No se puede comparar nuestro CNF contra el `1-cnf.txt` de referencia línea por línea, porque los nombres de las variables nuevas son arbitrarios (nuestro `T1` es su `L`). Lo que sí tiene sentido es comparar **comportamiento**: la prueba corre CYK con las dos gramáticas sobre una lista de cadenas y verifica que acepten y rechacen exactamente las mismas.

8. Discusión

8.1. Obstáculos encontrados

Programación dinámica sin variables mutables. Este fue el obstáculo conceptual principal. CYK se enseña con una matriz que se va llenando, y en Elixir no existen las variables mutables. La solución salió de observar una propiedad del algoritmo: CYK llena la tabla **por longitud creciente**, así que cuando se calcula una celda, todas las que necesita ya están escritas y solamente se leen. Nunca hay que modificar algo ya calculado. Eso convierte la matriz en un `Map` que se pasa como acumulador de un `reduce`, sin perder nada de eficiencia y sin trucos.

El reloj del sistema no alcanza para medir. El enunciado pide reportar el tiempo de validación. La primera versión usaba `:timer.tc` y reportaba `0.000 ms`. Investigando resultó que el reloj monotónico del sistema operativo tiene una resolución de unos 100 microsegundos en Windows, y una validación tarda unos 14. Es decir, el instrumento era más grueso que lo que se quería medir. La solución fue repetir la validación hasta superar un umbral de tiempo y dividir el total entre el número de corridas.

El calentamiento de la máquina virtual. Relacionado con lo anterior: la primera ejecución medía 1.3 ms y las siguientes 0.013 ms, cien veces menos. La causa es el compilador JIT de la BEAM, que optimiza el código recién después de ejecutarlo varias veces. Reportar la primera corrida habría dado un número engañoso. El esquema de promediar varias corridas resuelve también este problema.

Variables duplicadas en la binarización. La primera versión de la binarización creaba una variable nueva por cada cuerpo largo que encontraba. Como `( E )` aparece en tres reglas distintas, generaba `X1`, `X3` y `X5` con el mismo contenido `E T2`, dando 22 producciones donde el CNF de referencia tiene 20. Se agregó un registro de pares ya encadenados para reutilizarlos. Con eso el resultado coincide exactamente en tamaño y estructura con el de referencia.

La gramática del enunciado ya estaba en CNF. Al correr los cuatro pasos sobre la gramática del proyecto, ninguno cambió nada. No era un error: revisando regla por regla, todas son ya de la forma $A \to BC$ o $A \to a$, no hay épsilon, no hay unitarias y no hay símbolos inútiles. `VP -> cooks` parece unitaria pero no lo es, porque `cooks` es terminal, no variable. Por

eso agregamos la gramática aritmética de referencia, que sí ejercita los cuatro pasos de principio a fin.

Una ventana que colgaba el programa. El árbol se abre en una ventana aparte. La primera implementación usaba `cmd /c start`, y el programa se quedaba congelado hasta que uno cerrara el navegador. La razón es que el navegador heredaba la salida estándar del proceso y Elixir esperaba a que se cerrara. Se cambió a `explorer.exe`, que lanza el visor y devuelve de inmediato.

Una prueba mal escrita. Habíamos puesto *the cake eats* en la lista de frases que deberían rechazarse, y la prueba falló. Revisando, el programa tenía razón y nosotros no: la frase es sintácticamente correcta. Pasó a la lista de aceptadas, y terminó siendo el mejor ejemplo de la advertencia del enunciado.

8.2. Recomendaciones

- **Separar la lógica de la interfaz desde el principio.** Permitió escribir pruebas sin simular una terminal, y cambiar la presentación sin riesgo de romper los algoritmos.
- **Usar un ejemplo ya resuelto como prueba automática.** Tener `1.txt` y `1-cnf.txt` y poder verificar la equivalencia entre ambos dio mucha más confianza que revisar la salida a ojo.
- **Desconfiar de las mediciones de tiempo muy chicas.** Si el número sale 0 o varía cien veces entre corridas, el problema casi siempre está en el instrumento, no en el algoritmo.
- **Cuando una prueba falla, revisar primero la prueba.** En nuestro caso el código estaba bien y el error era nuestro entendimiento de la gramática.
- **Para una próxima versión**, se podría mostrar todos los árboles cuando la frase es ambigua, en vez de uno solo más la cuenta. La información ya está en los backpointers; solo falta recorrerlos exhaustivamente.

9. Conclusiones

El programa cumple lo que pide el enunciado: implementa el algoritmo de simplificación de gramáticas, implementa CYK, responde sí o no, reporta el tiempo de validación y construye el árbol de derivación.

Más allá de eso, el ejercicio dejó claro por qué la Forma Normal de Chomsky no es un requisito arbitrario: es la condición que hace posible atacar el problema con programación dinámica. Sin la restricción de dos símbolos por cuerpo, no hay dónde partir la subcadena y el algoritmo no existe.

También quedó claro que la elección de un lenguaje funcional, que al principio parecía un obstáculo para implementar programación dinámica, terminó siendo una ventaja. Los tres pasos de simplificación son cálculos de punto fijo y se escriben casi igual que en la pizarra, y la tabla de CYK resultó no necesitar mutación en ningún momento.

10. Referencias

- Hopcroft, J., Motwani, R. y Ullman, J. *Introduction to Automata Theory, Languages, and Computation.* 3ª edición.
- Aho, A., Lam, M., Sethi, R. y Ullman, J. *Compiladores: principios, técnicas y herramientas.* 2ª edición.